

Fallbacks in Microservice Failures

 systemdesignschool.io/fundamentals/fallbacks



Fallbacks

Circuit breakers detect persistent failures and stop calling failed services. But blocking requests leaves users with error pages. Fallbacks provide alternative responses when primary operations fail. The system degrades gracefully instead of breaking completely.

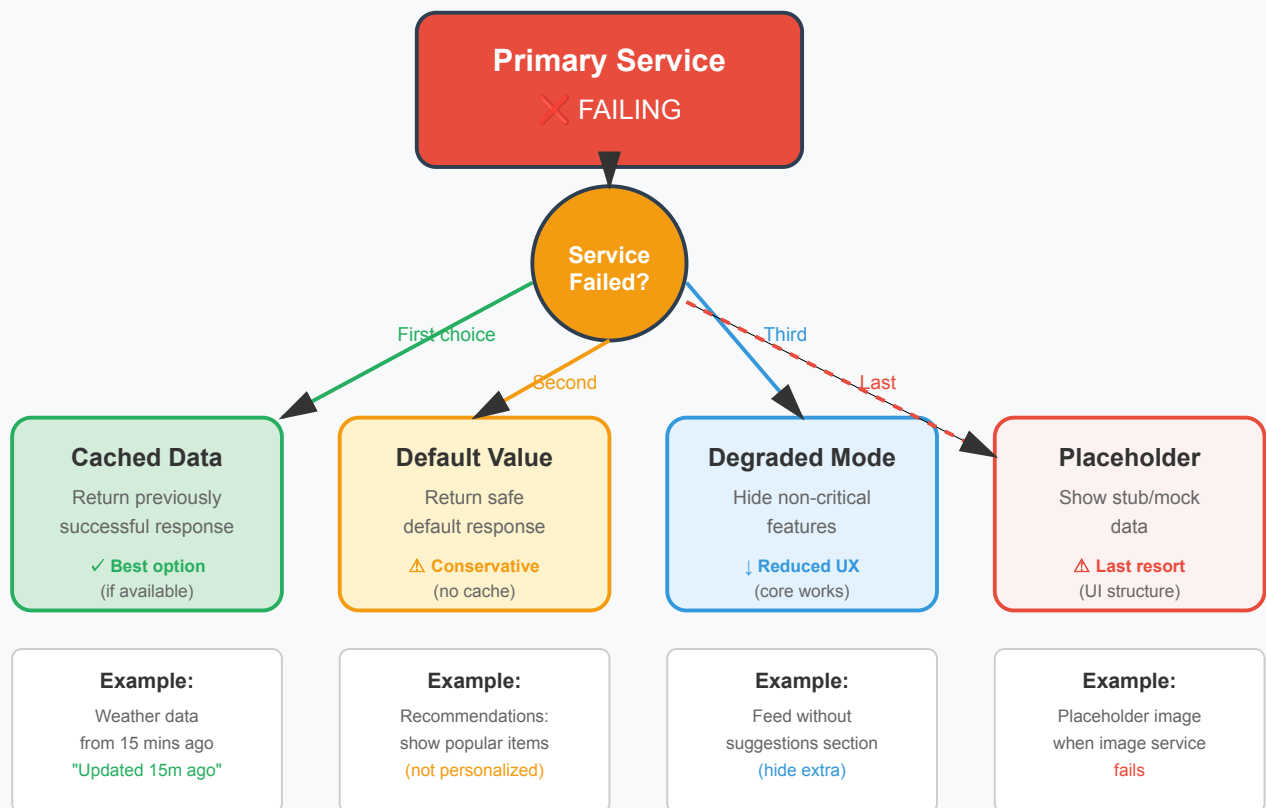
Why Fallbacks Matter

A product page displays item details, reviews, recommendations, and inventory status. The reviews service goes down. Without fallbacks, the entire page shows an error. The user cannot see the product or buy it, even though that functionality works. One failed dependency breaks the entire feature.

With fallbacks, the page loads without reviews. A message says "Reviews temporarily unavailable." The user sees the product details, checks inventory, and completes their purchase. Revenue continues. User experience degrades slightly but does not break.

Fallbacks turn catastrophic failures into minor inconveniences. They separate critical functionality from nice-to-have features. When non-critical services fail, the system continues operating with reduced features.

Fallback Strategies



Cached Responses

The simplest fallback returns previously cached data. A service caches successful responses with timestamps. When the primary service fails, return the cached version with a staleness indicator.

A weather service normally fetches live data from an external API. The API goes down. The service returns weather data from 15 minutes ago with a timestamp showing the age. Users get useful information instead of an error. Weather changes slowly enough that 15-minute-old data remains valuable.

Cache staleness must match the domain. Stock prices change every second. A 15-minute-old price is misleading. Return an error instead. User profiles change rarely. A day-old cached profile works fine as a fallback.

Set cache expiration appropriately. A 5-minute cache for product prices balances freshness with fallback coverage. A 24-hour cache for user avatars reduces load on the profile service while providing fallbacks for extended outages.

Default Responses

When cached data does not exist or is too stale, return a safe default value. The key is choosing defaults that never mislead users.

A recommendations service suggests products based on user history. The service fails. Return a default list of popular items instead. Users get recommendations even without personalization. The recommendations might not match their preferences, but they are still relevant products. Showing popular items is better than showing nothing.

A discount calculator determines pricing for enterprise customers. The calculator service fails. Return the full price rather than a default discount. Charging full price is safe. Applying an incorrect discount loses revenue or violates contracts. The conservative default protects the business.

Make defaults explicit. Do not return zero or empty string as a default unless that makes sense in context. If the discount calculator returns zero, the checkout process might interpret that as "free." Return an error code alongside the default so callers know they are getting fallback data.

Graceful Degradation

Disable non-essential features when their backing services fail. Core functionality continues while extras go offline.

A social media feed shows posts from friends, trending topics, and suggested accounts to follow. The suggestions service fails. Hide the suggestions section entirely. Users still see their friend's posts and trending topics. The feed works even though one enhancement is missing.

A video player supports multiple quality levels. The quality detection service fails. Default to standard definition instead of failing playback. Users can watch the video even if automatic quality selection is unavailable. Playback is more important than optimization.

Communicate degradation clearly. Show a banner: "Some features temporarily unavailable." Users understand the situation instead of wondering why things look different. They can retry later or contact support if needed.

Stubbed Responses

Return placeholder data that maintains UI structure without real information. This works when showing empty space would confuse users but real data is unavailable.

An e-commerce site shows product images, titles, prices, and descriptions. The image service fails. Show a generic product placeholder image. The page layout remains intact. Users see the title and price and can make purchasing decisions. Missing images are an annoyance, not a blocker.

A dashboard displays charts and graphs. The analytics service fails. Show the chart framework with a "Data temporarily unavailable" message where the graph would appear. The dashboard layout remains consistent. Other working charts still display. Users can check back later for the missing data.

Stubs must be clearly identifiable. A placeholder image should look obviously generic, not like actual product photography. Users should never mistake stubbed data for real data.

Combining Fallbacks With Circuit Breakers

Circuit breakers and fallbacks work together. When the circuit opens, the system immediately returns a fallback without attempting the failing call. This prevents wasting resources on calls that will fail.

A user profile service calls the achievements service for badges. The achievements service fails repeatedly. The circuit breaker opens. For the next 30 seconds, profile requests skip the achievements call entirely and return the profile without badges. After 30 seconds, the circuit goes half-open and tests a few requests. If achievements recovers, badges reappear. If not, the circuit reopens and fallback continues.

During an interview, discuss fallback strategies for the specific system you are designing. Identify which services are critical and which can degrade. Explain what users experience during each type of failure. Show that you understand the difference between catastrophic failures and acceptable degradation.